

Detail Technical Report: Cross Check of the Hardware Monitoring for the CMS Inner Tracker Upgrade (Ph2_ACF)

Marco Belotti

Phase-2 CMS Inner Tracker Group

Supervisors:

Simone Gennai

Giulio Bardelli

Athar Ahmad

CERN, September 2026

Abstract

The High-Luminosity LHC (HL-LHC) upgrade of the CMS Inner Tracker (IT) requires continuous hardware monitoring of its CROC (RD53B) readout chips and LpGBT optical-link devices, but the Ph2_ACF Data Acquisition (DAQ) framework currently records this telemetry only as unstructured text in its run LOG files. This report presents a real-time monitoring pipeline that bridges that gap, parsing Ph2_ACF LOG output as it is produced and exposing it as structured, continuously-refreshed metrics to Prometheus and Grafana. The pipeline is organized as three decoupled stages – LOG-to-CSV extraction, disk-bounded chunking, and Prometheus export – tied together by a shared configuration and data-schema layer, so that any one stage can later be replaced without touching the others. Beyond real-time streaming, the system performs on-the-fly physical-unit conversion of CROC registers and per-chip calibration of non-calibrated LpGBT ADC channels, and guarantees a bounded disk footprint through an atomic, zero-data-loss truncation mechanism that never discards a measurement before it has reached Prometheus. Each monitored quantity is tracked and evicted independently. The pipeline was validated on three different laboratory setups, combining CROC and LpGBT devices in different configurations, and further applied also to an exploratory studying of the VTRx+ optical-power monitoring. HL-LHC is expected to have the structured CSV production done natively by Ph2_ACF or the DTC software, this project is intended primarily as a cross-check of the monitoring parameters and a small-size test, rather than as that system itself.

Contents

1	Phase-2 CMS Inner Tracker: Detector and Read-Out Chain	3
1.1	The sensors	3
1.2	The CROC	4
1.2.1	Monitoring Functions	4
1.3	Opto-Electrical Read-Out Chain	5
1.3.1	LpGBT Monitoring	5
2	Acquisition and Control Framework: Ph2_ACF	6
3	Monitoring Test Setup	8
4	Project Objective and Implementation	10
4.1	Real-Time Hardware Monitoring Pipeline	11

5	csv_schema.py	12
6	config_loader.py	13
6.1	The monitoring.ini file	13
7	log_to_csv.py	14
7.1	Process orchestration	14
7.2	Run-number lifecycle management	14
7.3	Real-time LOG parsing	14
7.4	CROC vs. LpGBT: two different telemetry shapes	15
7.5	Unit conversion at the point of ingestion	15
7.6	Duplicate suppression	15
7.7	Output: one append-mode CSV, not a chunked writer	16
8	csv_chunker.py	16
8.1	Tailing the Full CSV safely	16
8.2	Chunking by monitoring cycle, not by size or time	16
8.3	Bounding the Full CSV's size, without racing the writer	16
9	exporter.py	17
9.1	The label set is the identity of a time series	17
9.2	Consume-then-delete, on every scrape	17
9.3	Per-series staleness, not global staleness	17
10	Other Applications	18
11	Results: Grafana Plots	18
A	CSV Data Sizing	21
B	Prometheus YAML file	21
C	Off-line LOG Parsing	21
D	GitHub Repository	22

1 Phase-2 CMS Inner Tracker: Detector and Read-Out Chain

During the Long Shutdown 3, the CMS experiment at CERN will be upgraded to achieve the goal of High Luminosity of the LHC accelerator (HL-LHC), that will guarantee an higher amount of statistics and new possible discoveries. Due to the hard radiation environment of HL-LHC, the present CMS pixel detector will be replaced, with a new Inner Tracker. The detector is composed of silicon pixel sensors and the readout opto-electric chain.

1.1 The sensors

These new devices needs high radiation tolerance, a low material budget, high granularity to improve the spatial resolution, high single-hit detection efficiency, and minimal power dissipation. The design will reach a pseudo-rapidity coverage up to $|\eta| = 4.0$, thanks to three sub-detectors: the Tracker Barrel Pixel (TBPX), the Tracker Forward Pixel (TFPX) and the Tracker Endcap Pixel (TEPX). Each part will be composed constructed from hybrid pixel modules with two (double) or four (quad) readout chip (CROCs). To mitigate the issue of radiation damage - in particular for the layer closest to the interaction point - 3D sensor has been developed. These shows better performances with respect to the planar ones. These sensors have columnar electrodes penetrating the bulk perpendicular to the surface. The consequences are: lower depletion voltages and, consequently, lower power dissipation, as well as shorter collection path (and so reduced trapping probability after irradiation) [1].

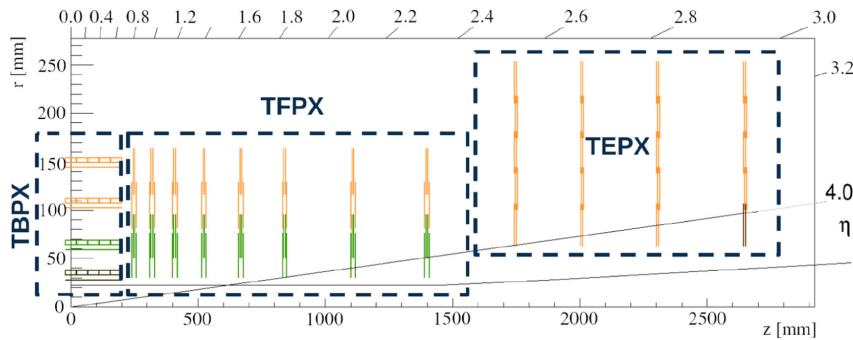


Figure 1: A slice of a quarter of the CMS IT layout. In green the dual modules and in orange the quad modules.

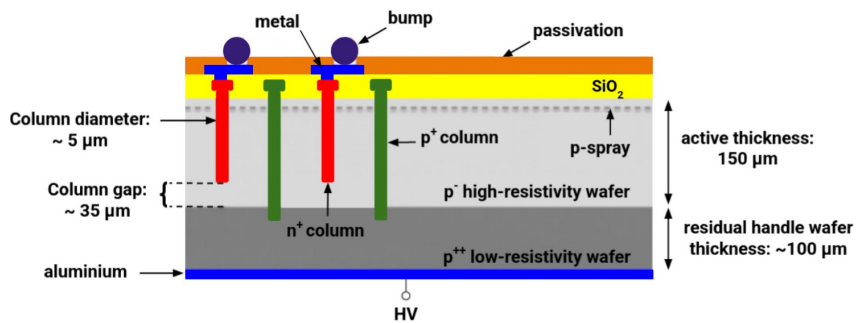


Figure 2: Cross-section of the 3D pixel silicon sensor. Notice the high and low resistivity layers and the p^+ and n^+ columns.

The 3D sensor is composed of 2 p -type layers, one with low and one with high resistivity, bounded together. The active region of the sensor is the high resistivity layer ($3 \text{ k}\Omega\text{cm}$ and $4.5 \cdot 10^{12} \text{ cm}^{-3}$ doping concentration), while the low resistivity one provides mechanical support

and ohmic contact. The n^+ columns provides the electronic contact of the sensor to the ASIC (Application-Specific Integrated Circuit) through the bumps. Each pixel cell features a single n^+ columnar electrode at the center and four p^+ columns at the corners. The sensor size is $22 \times 17 \text{ mm}^2$ and the thickness is about $150 \text{ }\mu\text{m}$. This guarantees a Most Probable Value of collected charge for a MIP of about 10.200 electrons [1].

1.2 The CROC

The CMS Read-Out Chip (CROC) is the result of a collaboration (RD53) that has produces readout chips for CMS and ATLAS experiment for the HL-LHC phase, that can sustain very high rates in an extremely hostile radiation environment. RD53B is a pixel readout chips framework that can be instantiated into different size physical chips. The chip sensitive area (placed on top of the chip) is arranged as a matrix of pixels of $50 \times 50 \text{ }\mu\text{m}^2$. The peripheral circuit is placed at the bottom of the chip and contains all global analog and digital circuit needed to bias, monitor and readout the chip [2]. The main physical characteristics of the chips are reported in this table:

Parameter	CMS
Pixel bump pitch	$50 \times 50 \text{ }\mu\text{m}^2$
pixel rows (H)	336
pixel columns (W)	432
core rows	42
core columns	54
Chip width	21.654 mm
Chip height	18.622 mm

Table 1: Chip size for the CMS chips.

The readout chips are powered at a minimum of 1.4 V, and never more that 2.0 V: 1.6 V should be the typical settings. In the monitoring phase, the corresponding VDDA and VDDD values should produce approximately 1.2 V [2].

1.2.1 Monitoring Functions

To guarantee reliable detector operation and real-time diagnostic capability within the HL-LHC operation, the CROC (RD53B) integrates a Monitoring clock, that enables digitization and readout of internal parameters, such as temperature, total ionizing dose, voltages, and currents, from different parts of the chip and even external ones [2]. This allows internal parameters to be streamed through the DAQ pipeline, following the normal output path (as data). Monitoring can be performed at any time, also during data-taking. The Monitoring block is composed of 2 sub-block: the analog current multiplexer followed by the voltage multiplexer (MUX) and the 12-bit Analog-to-Digital Converter (ADC). The first monitors the voltages, while the second digitizes the selected MUX channels, the temperatures, and the radiation sensor outputs. The other monitored quantities are temperature and total radiation:

- **Resistive Temperature Sensors:** Integrated polysilicon/NTC resistive elements placed along the peripheral region of the chip. The polysilicon resistors are designed to measure the temperature difference between top and bottom of the chip. These resistive sensors can precisely measure temperature changes, but not absolute temperature, while the NTC can.
- **Radiation Sensors:** The Bipolar Junction Transistor (BJTs) are very sensitive to both ionizing dose and displacement damage. Even if born as temperature sensors, in the RD53B are used as radiation sensors

1.3 Opto-Electrical Read-Out Chain

The read-out chain is designed to have high throughput, reliability over long paths and reduced passive material. To achieve that the largest portion of the chain consists of optical fibers that allow for data exchange to and from the DTC (Data Trigger and Control) board.

The read-out system consists of the following chain:

- Module (dual or quad RD53B chip - CROC)
- PortCard: LpGBT and VTRX+
- Backend (FPGA - FC7 in the Lab, DTC in CMS)

Define as *Uplink* stream the transmission of data (from module to backend) and *Downlink* the transmission of command (from backend to the module).

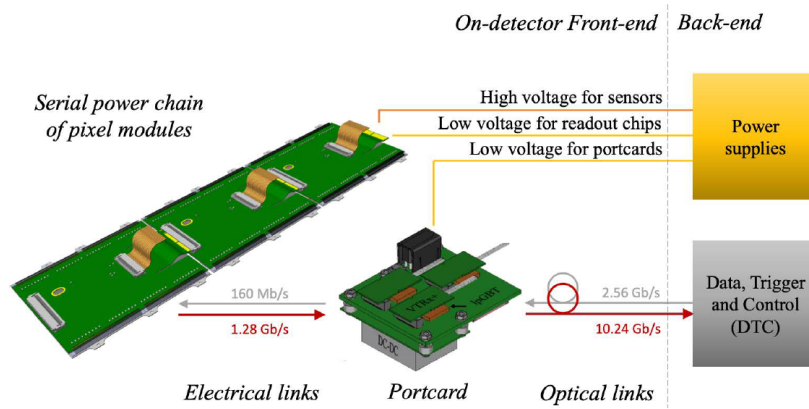


Figure 3: A sketch of the Read-Out chain. *Image courtesy of Giulio Bardelli [3].*

The connection of the chain are the following:

- **Module - PortCard:** Electrical connection - E-links (low mass twisted pair) - via Aurora Protocol. $(1 - 6) \times 1.28$ Gbps (Downlink) and 160 Mbps (Uplink).
- **LpGBT - VTRX+** - Inside the PortCard: High speed links via PCB.
- **PortCard - FC7 FPGA:** Optical fibres. 10.24 Gbps (Downlink) and 2.56 Gbps (Uplink).

The PortCard provides the conversion from electrical signal to optical one. In this way the E-links need to cover no more than 2 m in length (from module to PortCard). The conversion is made by the Low-power Giga Bit Transceiver (LpGBT) and the Versatile Link R+ (VTRX+), that is actually the laser driver to which the fibers are attached. The powering of the PortCard is ensured by the DC-DC converter mounted on the mezzanine (bPOL12V). In each PortCard there are 3 LpGBTs, each one connected to a VTRX+. The VTRX+ registers are directly initialized by the LpGBT, via I2C protocol. The optical fibres from the VTRX+ are connected to the FC7 through a SFP FMC [3].

1.3.1 LpGBT Monitoring

Because the VTRX+ optical transceiver is directly interfaced with and initialized by the LpGBT, VTRX+ monitoring is integrated directly into the LpGBT monitoring infrastructure [4]. The LpGBT analog block provides full diagnostic capabilities for both internal chip parameters and external board-level signals via an integrated sensing network.

The core components of the LpGBT analog peripheral architecture are similar to the CROC one and comprise:

- **10-bit SAR ADC & Differential Gain Stage:** A fully differential 10-bit Successive Approximation Register (SAR) ADC digitizes analog inputs (voltages, currents, temperatures, and optical power). An integrated programmable gain stage placed before the ADC core allows differential amplification factors of $2\times$, $8\times$, $16\times$, and $32\times$ (halved for single-ended measurements), optimizing dynamic range for small differential signals. This performs the digitization of the physical monitored quantity (temperature, voltage, current, optical power...).
- **16-Channel Analog Multiplexer (MUX):** With the Multiplexer the input physical signals are routed to the ADC. MUX consists of:
 - *8 External Channels (ADC0–ADC7):* Pins reserved for board-level sensors and external voltage rails.
 - *8 Internal Channels:* Dedicated to internal supply voltage monitors (V_{DD} , V_{DDA} , V_{DDTX} , V_{VDDRX}), internal temperature sensors, ground references, and internal DAC outputs.

PortCard Sensor Mapping On the PortCard, the external ADC channels (ADC0–ADC7) across the different LpGBT are allocated to specific monitoring tasks:

- Two connections dedicated to bPOL DC-DC converter temperature sensors.
- One connection for the VTRX+ temperature sensor.
- One connection for the VTRX+ Received Signal Strength Indicator (RSSI) to monitor optical power.
- Three voltage monitoring channels: VTRX+ power rail ($\sim 1.25\text{ V}$), LpGBT power rail ($\sim 2.5\text{ V}$), and primary input voltage ($\sim 10\text{ V}$).
- One channel dedicated to the internal LpGBT temperature sensor.
- Additionally, only one LpGBT allocates an external channel to monitor the global PortCard temperature.

Calibration In order to improve the accuracy of the reference voltage generator, ADC, current and voltage DACs, internal temperature sensor as well as VDD monitors, these circuits are individually characterized for each device at two temperatures during production testing. Calibration data is provided to users that allows trimming the analog chain against chip-to-chip variations as well as temperature-dependent effects. The internal eFuse is used to identify each device [4].

2 Acquisition and Control Framework: Ph2_ACF

All the chip (from RD53B to LpGBT and VTRX+) must be calibrated for optimal performances. For this purpose the software Ph2_ACF has been developed. Firstly Ph2_ACF performs the initialization of all the registers and then it can run different calibrations and scans, and also manage the hardware monitoring [5].

The main entry point to run anything is with the CMSITminiDAQ executable. It is configured through one XML file that contains the structure of the setup where you are running the calibration: detector hierarchy, readout chip and FC7 register configuration, the calibration/scan configuration and the monitoring settings. The detector hierarchy has this structure:

Board $\rightarrow$ Optical Group: PortCard (LpGBT settings) $\rightarrow$ Hybrid: module $\rightarrow$ Chip (CROC settings)

In the chip settings, you can assign a value to a specific register. Some of them are global and some are dedicated to specific calibrations.

In the XML file also loads the `txt` files containing the name of the registers of all the chips (RD53s and LpGBTs), and their default values.

Monitoring: The periodic readout of monitored quantities like voltages, temperature, currents... are obtained enabling the dedicated section in the XML file. The `MonitoringSleepTime` can be tuned in the XML file depending on the needs. It expresses the pause time between the end of a monitoring and the beginning of the next. In this section each register to be monitored can be enabled or disables individually. The monitoring is performed for all the components of the setup in a precise sequence. The same register is read for all the chips of that type (RD53s or LpGBTs), in the order specified in the detector hierarchy settings. The monitoring registers are read in the order specified in the monitoring settings section.

```
<MonitoringSettings>
  <Monitoring type="RD53B" enable="1" silentRunning="0">
    <MonitoringSleepTime> 1000 </MonitoringSleepTime>
    <MonitoringElement device="RD53" register="VINA" enable="0"/>
    <MonitoringElement device="RD53" register="VDDA" enable="0"/>
    <MonitoringElement device="RD53" register="ANA_IN_CURR" enable="0"/>
    <MonitoringElement device="RD53" register="VIND" enable="0"/>
    <MonitoringElement device="RD53" register="VDDD" enable="0"/>
    <MonitoringElement device="RD53" register="DIG_IN_CURR" enable="0"/>
    <MonitoringElement device="RD53" register="Iref" enable="0"/>
    <MonitoringElement device="RD53" register="POLY_REL_TEMPSSENS_TOP" enable="0"/>
    <MonitoringElement device="RD53" register="POLY_REL_TEMPSSENS_BOTTOM" enable="0"/>
    <MonitoringElement device="RD53" register="POLY_ABS_TEMPSSENS_TOP" enable="0"/>
    <MonitoringElement device="RD53" register="POLY_ABS_TEMPSSENS_BOTTOM" enable="0"/>
    <MonitoringElement device="RD53" register="TEMPSSENS_ANA_SLDO" enable="0"/>
    <MonitoringElement device="RD53" register="TEMPSSENS_DIG_SLDO" enable="0"/>
    <MonitoringElement device="RD53" register="TEMPSSENS_CENTER" enable="0"/>
    <MonitoringElement device="RD53" register="INTERNAL_NTC_REL" enable="0"/>
    <MonitoringElement device="RD53" register="INTERNAL_NTC_ABS" enable="0"/>

    <MonitoringElement device="LpGBT" register="ADC0" enable="1"/>
    <MonitoringElement device="LpGBT" register="ADC1" enable="1"/>
    <MonitoringElement device="LpGBT" register="ADC2" enable="1"/>
    <MonitoringElement device="LpGBT" register="ADC3" enable="1"/>
    <MonitoringElement device="LpGBT" register="ADC4" enable="1"/>
    <MonitoringElement device="LpGBT" register="ADC5" enable="1"/>
    <MonitoringElement device="LpGBT" register="ADC6" enable="1"/>
    <MonitoringElement device="LpGBT" register="ADC7" enable="1"/>
    <MonitoringElement device="LpGBT" register="EOM_DAC" enable="0"/>
    <MonitoringElement device="LpGBT" register="VDDIO" enable="1"/>
    <MonitoringElement device="LpGBT" register="VDDTX" enable="1"/>
    <MonitoringElement device="LpGBT" register="VDDRX" enable="1"/>
    <MonitoringElement device="LpGBT" register="VDD" enable="1"/>
    <MonitoringElement device="LpGBT" register="VDDA" enable="1"/>
    <MonitoringElement device="LpGBT" register="TEMP" enable="1"/>
    <MonitoringElement device="LpGBT" register="VREF/2" enable="1"/>
    <MonitoringElement device="LpGBT" register="PORTCARD_NTC_ABS" enable="0"/>
    <MonitoringElement device="LpGBT" register="VTRX_NTC_ABS" enable="0"/>
    <MonitoringElement device="LpGBT" register="PUSMStatus" enable="1"/>
  </Monitoring>
</MonitoringSettings>
```

Figure 4: An example of the monitoring settings in the XML file. Notice that only the LpGBT registers are enabled for monitoring.

Running Ph2_ACF: To run CMSITminiDAQ you have to specify the following command:

```
CMSITminiDAQ -f configuration_file.xml -c calibration_name
```

Where `configuration_file.xml` is the XML configuration file and `calibration_name` is the name of the calibration to run (such as `physics`, `vtrx`, `pixelalive`, `scurve`...). The output of the calibration is printed at terminal and saved in the LOG file. At the end of the calibration also the XML and all the `txt` files are saved. In the LOG file are saved all the operation performed by Ph2_ACF: reading the XML file, connecting to all the register and reading their eFuse (the unique

identifier of each electronic device), initializing all their registers, the values of the monitored register for the specific device, the progress and the results of the calibration.

```

09:54:22[I] === Initializing communication to Low-power Gigabit Transceiver (LpGBT): 1 ===
09:54:22[I] --> Configured up and down link mapping in firmware
09:54:22[I] LpGBT version: LpGBT-v2
09:54:22[I] LpGBT PUSH status: READY
09:54:23[I] Initializing registers of LpGBT: 1
09:54:23[I] --> Done
09:54:23[I] LpGBT chip info: Tx Data Rate = 10 Gbit/s; TxEncoding = FEC5; LpGBT Mode = Transceiver
09:54:23[I] Checking DLL status of LpGBT: 1
09:54:23[I] --> DLL status of Rx Group 4 is 0xC0
09:54:23[I] --> DLL status of Rx Group 0 is 0xC0
09:54:23[I] --> Done
09:54:24[I] FuseID from LpGBT OpticalGroup ID 1 on Board ID 0: 0x52AE68FD
09:54:24[I] Loading calibration data for LpGBT on Board 0 OpticalGroup 1 with Fuse ID 0x52AE68FD
09:54:25[W] --> Calibration data available for LpGBT on Board ID 0 OpticalGroup ID 1 with Fuse ID 0x52AE68FD
09:54:25[I] Calibrating LpGBT Vref...
09:54:33[I] --> Done
09:54:34[I] Phase aligning Rx Group: 4
09:54:34[I] --> Group 4 LOCKED
09:54:34[I] --> Channel 0 has phase 9
09:54:34[I] Phase aligning Rx Group: 0
09:54:34[I] --> Group 0 LOCKED
09:54:34[I] --> Channel 0 has phase 6
09:54:34[I] === LpGBT chip 1 configured ===

09:55:15[I] --> LpGBT voltage measurement from power supply VDDTX is 1.275 V
09:55:15[I] Reading monitored data for [board/opticalGroup = 0/1]
09:55:15[I] --> LpGBT voltage measurement from power supply VDDTX is 1.277 V
09:55:15[I] Reading monitored data for [board/opticalGroup = 0/2]
09:55:16[I] --> LpGBT voltage measurement from power supply VDDTX is 1.280 V
09:55:16[I] Reading monitored data for [board/opticalGroup = 0/0]
09:55:17[I] --> LpGBT voltage measurement from power supply VDDR1 is 1.278 V
09:55:17[I] Reading monitored data for [board/opticalGroup = 0/1]
09:55:17[I] --> LpGBT voltage measurement from power supply VDDR1 is 1.279 V
09:55:17[I] Reading monitored data for [board/opticalGroup = 0/2]
09:55:18[I] --> LpGBT voltage measurement from power supply VDDR1 is 1.282 V
09:55:18[I] Reading monitored data for [board/opticalGroup = 0/0]
09:55:19[I] --> LpGBT voltage measurement from power supply VDD is 1.278 V
09:55:19[I] Reading monitored data for [board/opticalGroup = 0/1]
09:55:19[I] --> LpGBT voltage measurement from power supply VDD is 1.282 V
09:55:19[I] Reading monitored data for [board/opticalGroup = 0/2]
09:55:20[I] --> LpGBT voltage measurement from power supply VDD is 1.280 V
09:55:20[I] Reading monitored data for [board/opticalGroup = 0/0]
09:55:21[I] --> LpGBT voltage measurement from power supply VDDA is 1.275 V
09:55:21[I] Reading monitored data for [board/opticalGroup = 0/1]
09:55:21[I] --> LpGBT voltage measurement from power supply VDDA is 1.279 V
09:55:21[I] Reading monitored data for [board/opticalGroup = 0/2]
09:55:22[I] --> LpGBT voltage measurement from power supply VDDA is 1.282 V
09:55:22[I] Reading monitored data for [board/opticalGroup = 0/0]
09:55:23[I] --> LpGBT temperature measurement 33.672 C
09:55:23[I] Reading monitored data for [board/opticalGroup = 0/1]
09:55:23[I] --> LpGBT temperature measurement 32.538 C
09:55:23[I] Reading monitored data for [board/opticalGroup = 0/2]
09:55:24[I] --> LpGBT temperature measurement 34.716 C

```

Figure 5: Example of Ph2_ACF output. On the Top: The initialization of an LpGBT, notice the detection of the eFuse. On the Bottom: An example of a Physics running and the output of some LpGBT monitored values.

This project focused mainly on the LpGBT monitoring - but also CROC monitoring has been developed. Some dedicated setup were used to test the code and check its performances.

3 Monitoring Test Setup

The setups used in this project are the following:

- **Setup 1:** Dual Chip (2×1 RD53Bv2) read by one LpGBT
- **Setup 2:** Three LpGBTs (and a “passive” Dual Chip)
- **Setup 3:** Two LpGBTs (and a “passive” Dual Chip)

Notice that the Dual Chip is a L1 module and will be a component of the CMS Tracker Barrel Pixel Detector (TBPX). All the setup are operated using a Peltier Cell to cool down the chips and a fan to dissipate the heat. The PortCard is placed in front of the fan to receive an indirect cooling.

Because in the XML file settings you can choose to test only a part of the readout chain (with the parameter `chain2Test=0, 1, or 2`), in the first setup all the chain was configured and tested, from backend (FC7) to frontend (RD53B). In this part of the project also the CROC monitoring has been studied.

For the other setups, all the LpGBTs in the PortCard where optical connected to backend (or only two of them), but only one of them was connected to the dual chip. By setting the `chain2Test=1`, we have been able to perform only the chain testing of the three (two) LpGBT to the backend, disabling the initialization of the RD53Bv2 chips (`SkipInitRD53=1`). So for these setups only the monitoring of all the LpGBTs connected has been performed.

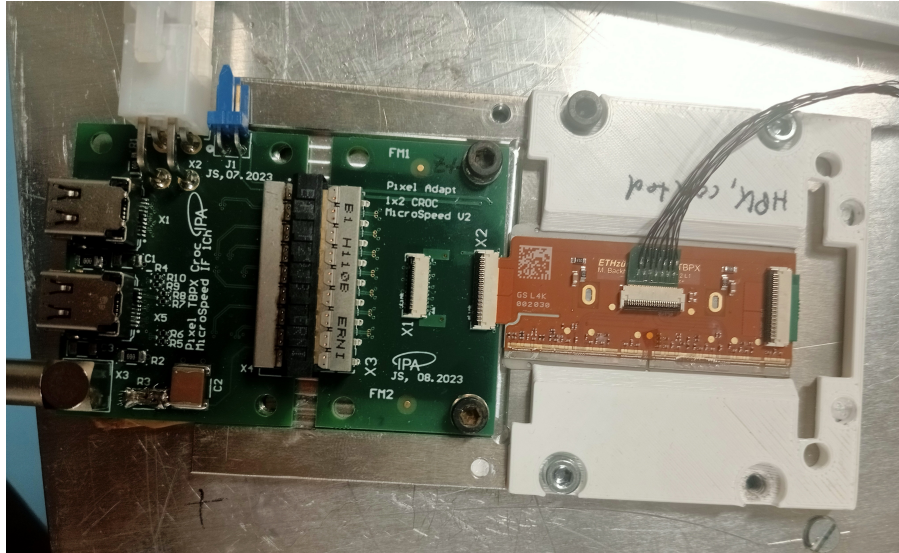


Figure 6: The Dual Chip (2×1 RD53Bv2) used in all the setups. Notice on the right the two chips and the E-links (twisted pairs) towards the LpGBT, and on the left the adapter board for powering.



Figure 7: The two PortCard sides in the first setup configuration. On the Left: The 3 LpGBTs, notice that the central one is covered by the VTRX+, from which the optical fibre comes out. In the other setups also the other LpGBTs are connected to the VTRX+, even if only the central is linked to the CROC. On the Right: The input E-links from the CROC, on the left notice the mezzanine bPOL12V DC-DC for powering.

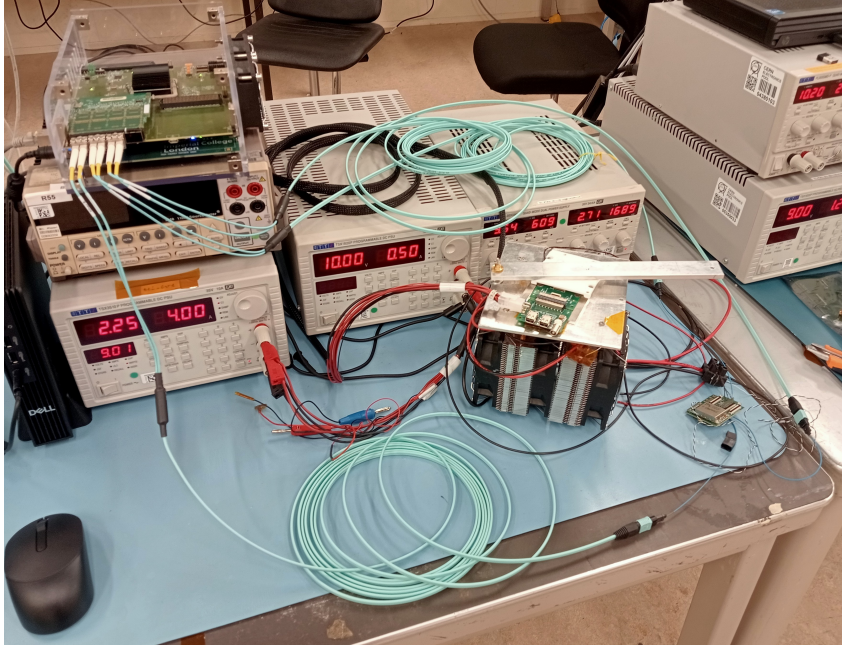


Figure 8: The complete second setup. We can see the CROC module over the Peltier Cell and the fan (in the middle), the PortCard (on the right) and the FC7 FPGA (on the left). Notice the three optical fibers that comes out from the PortCard (one for each LpGBT) and their connection in the FC7 board. We can also notice the powering: 10 V for the portcard (that reduced to 2.5 V for LpGBT and 1.25 V for VTRX+ thanks to DC-DC mezzanine) and 4 A for the chips (a 2 A current per chip is required for operations).

4 Project Objective and Implementation

My project focused on the possibility to extract the monitoring data (temperatures, voltages, currents...) from the LOG file produced by Ph2_ACF, filling a CSV file with them, expose metrics to the live-time database scraping (Prometheus) and produce monitoring time series with an external application (Grafana). My project solves this specific problem: Ph2_ACF writes hardware monitoring data as unstructured text into the LOG file during a run, but Prometheus and Grafana requires structured, continuously-refreshed numeric time series.

In the HL-LHC operations, CMS Inner Tracker would not use the pipeline I developed, because the CSV file will be directly produced by Ph2_ACF or by the DTC software. However, this project take the shape of a cross-check for all the monitoring-related parameters, and tries to achieve all the requirements of the real software monitoring infrastructure, that include:

1. **Low Latency Streaming:** Processing of DAQ monitoring streams and delivering the parsed metrics to Prometheus/Grafana in a (reasonable) short time (order of minutes).
2. **Robust Memory Bounds:** Preventing disk saturation during long DAQ runs.
3. **On-the-Fly Calibration:** Real-time conversion of CROC register values and calibration of non-calibrated LpGBT ADC channels, if necessary.
4. **Decoupled Modular Architecture:** Clear separation between (i) extraction of monitoring data and production of CSV file, (ii) data chunking, and (iii) Prometheus metric exposure, to allow code generalization and bug tolerance.

4.1 Real-Time Hardware Monitoring Pipeline

Architecture Overview The monitoring pipeline I implemented is organized into a three-stage decoupled architecture, each stage with one clear responsibility. A shared `monitoring.ini` configuration file and two shared modules (`config_loader.py`, `csv_schema.py`) tie the three stages together so they agree on file paths and data layout without duplicating that information in three places.

Figure 9 illustrates the data flow from the Ph2_ACF DAQ environment through the processing stages down to the visualization layer and table 2 the key responsibilities of each module of the pipeline.

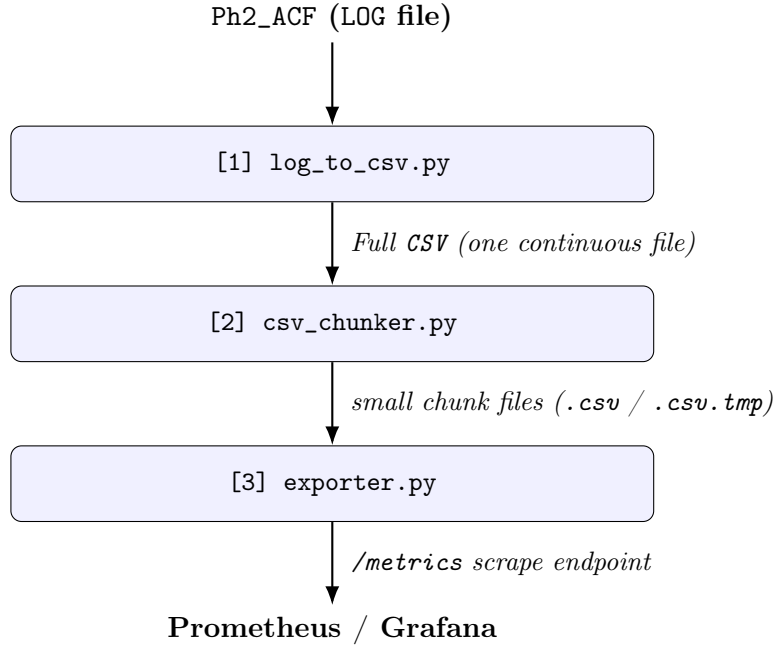


Figure 9: Decoupled three-tier pipeline architecture for real-time hardware monitoring.

Module File	Stage	Key Responsibilities & Mechanisms
<code>csv_schema.py</code>	Infrastructure	Contains the unique field data header for all storage stages and the monitored register for the LOG file parsing.
<code>config_loader.py</code>	Infrastructure	Centralizes configuration with <code>monitoring.ini</code> , manages I/O paths, and CLI fallback defaults.
<code>log_to_csv.py</code>	Stage 1: Parsing	Tail-reads the Ph2_ACF LOG files, parses hardware hierarchies, computes CROC physical values, applies LpGBT ADC calibration and fills the Full CSV.
<code>csv_chunker.py</code>	Stage 2: Batching	Groups CSV rows into atomic chunks based on monitoring-cycle; manages dynamic Full CSV truncation after <code>max_mb</code> .
<code>exporter.py</code>	Stage 3: Export	Atomically consumes chunk CSVs exposing them to <code>Prometheus</code> , deletes the scraped chunks, and evicts stale metrics after <code>stale_after</code> seconds.

Table 2: Functional responsibilities and operational mechanisms of each module of the software stack.

This separation exists for three concrete reasons, each addressed by a specific design choice documented in its section below:

1. **Prometheus scrapes by pulling a /metrics endpoint on a timer** – it cannot itself watch a LOG file, so something has to bridge push-style LOG lines into pull-style metrics.
2. **A single ever-growing CSV is disk-unsafe** – `csv_chunker.py` exists specifically to bound its size without losing data or racing the writer that is still appending to it.
3. **`log_to_csv.py`'s only job in the future is to be replaceable** – if `Ph2_ACF` is ever modified to write CSV directly instead of a LOG file, only stage 1 needs to change; stages 2 and 3 are entirely decoupled from *how* the Full CSV gets produced, only that it does.

In the following sections I'll describe in detail the purpose, design and characteristics of each code module.

5 `csv_schema.py`

Purpose: the single source of truth for the CSV column layout (`FIELDNAMES`) and the register-name used by the LOG parser.

Why it is separate: `FIELDNAMES` is read by the `log_to_csv.py` – that writes it – and by the `csv_chunker.py` – that reads and re-writes it in the CSVs chunks (and also in the optional debug one). Keeping one copy imported everywhere, guarantee consistency across all ingestion stages, and means an easily schema changing.

The structure of the CSV is build to be hierarchical (as explained in section 2) and to reflect the detector structure. These are the 14 fields used:

```
BOARD, OPTICAL_GROUP, PORTCARD_ID, LpGBT_EFUSE, HYBRID_ID, CHIP,
CHIP_EFUSE, MODULE_ID, DATE, TIME, REGISTER, VALUE, ERROR, UNIT
```

This is supposed to be also the structure of the final CSV file produced by the DTC, during CMS HL-LHC operation.

What else it contains:

- `CROC_REGISTERS` / `LPGBT_ADC_REGISTERS` / `LPGBT_VOLTAGE_REGISTERS` – the lists of register names that `log_to_csv.py` searches in each LOG line, separated by which hardware component they belong to (CROC vs. LpGBT).
- `PROBLEMATIC_LPGBT_ADC_REGISTER` – a dictionary, keyed by LpGBT eFuse, of which ADC channels on *that specific LpGBT* lack the calibration and therefore need the software-side conversion implemented in `log_to_csv.py`'s `adc_raw_to_voltage()`¹.
- `LPGBT_ADC_CONNECTION` – documents which physical quantity each ADC channel number corresponds to per chip variant (temperature, voltage, RSSI, etc.). *To be implemented.*

This piece of code cannot be run: it is pure data. In this way it can be reviewed and extended (e.g. adding a new chip's eFuse to the non-calibration list) without touching any control flow.

¹Pay attention: it's necessary to enable the LOG-printing `WARNINGS` from the `PH2_ACF` logger, to ensure that uncalibrated ADC (signed as `WARNINGS` lines) are directed to LOG files rather than unbuffered stdout

6 config_loader.py

Purpose: lets all three scripts read their default settings from one shared `monitoring.ini` file instead of requiring identical command-line flags typed out separately for each of the three long-running processes.

Design: a two-phase argument parse.

1. `load_config()` first runs a minimal `argparse` that looks *only* for `--config`, to determine which `.ini` file to load – without yet knowing the full set of arguments each specific script needs.
2. It hands back a `ConfigParser` object plus the resolved path.
3. Each script then builds its *real* `argparse` parser, using `cfg` to populate each flag's default value. Any value can still be overridden with a plain CLI flag for each code launched – the config file only changes what happens when a flag is *omitted*.

6.1 The monitoring.ini file

This is the single file all three scripts read from, setting the default values for each `--flag`. Its four sections mirror the pipeline's own structure: `[paths]` holds the two directories every other value is defined relative to; `[log_to_csv]`, `[csv_chunker]`, and `[exporter]` each hold the settings specific to that one stage, using the same key names as that script's own `--flag` options.

```
# Shared configuration for the monitoring pipeline:
# log_to_csv.py -> csv_chunker.py -> exporter.py
#
# Any value here can still be overridden with the matching --flag on the
# command line for a one-off run without editing this file.
#
# To use a different config file, pass --config /path/to/other.ini to any of the
# three scripts, or set the MONITORING_CONFIG environment variable.

[paths]
# Root of the Ph2_ACF installation (contains settings/ and test/)
ph2_acf_dir = /home/bootcamp/marco/Ph2_ACF_1/Ph2_ACF/

# Base directory everything else in this file is derived from:
# output_csv_dir/<output_csv_name> <- log_to_csv.py writes here
# output_csv_dir/chunks <- csv_chunker.py writes here, exporter.py reads here
output_csv_dir = /home/bootcamp/marco/code/MonitoringCSV

[log_to_csv]
calibration = physics
output_csv_name = monitoring_FULL.csv
timeout = 300
heartbeat_seconds = 120
log_time = false
terminal_ph2_acf = false
no_launch = false

[csv_chunker]
max_mb = 0.01
# Relative to [paths] output_csv_dir, same as output_csv_name above
debug_csv = monitoring_DEBUG.csv

[exporter]
stale_after = 150
```

Notice in particular that:

- [paths] `output_csv_dir` is deliberately the only place the shared base directory is ever written down. `csv_chunker.py`'s `--input_csv` and `--chunk_dir` defaults, and `exporter.py`'s `--csv_dir` default, are all *derived* from it (`{output_csv_dir}/chunks`, etc.) rather than being separate hard-coded values – so the two consumer scripts can never silently drift out of agreement with each other about where the chunk directory actually is.
- [log_to_csv] `output_csv_name` is read by *both* `log_to_csv.py` (to know what to name the file it writes) and `csv_chunker.py` (to know what to tail).

7 `log_to_csv.py`

Purpose: stage 1 of the pipeline. Optionally launches CMSITminiDAQ itself, then parses its LOG output line-by-line in real time, converting free-text telemetry lines into structured rows appended to one continuously-growing “Full CSV.”

This is the largest and most domain-specific file, so it is worth breaking into its sub-responsibilities.

7.1 Process orchestration

When run without `--no_launch`, the script builds and runs the exact shell sequence a human would type interactively, as described in section 2:

```
cd <Ph2_ACF_dir> &&
source setup.sh &&
cd test/ &&
CMSITminiDAQ -f <configuration_file.xml> -c <calibration>
```

via `subprocess.Popen`, using `preexec_fn=os.setsid` so the child lives in its own process group. This matters for clean shutdown: when monitoring ends – whether by reaching the LOG's end marker, hitting the inactivity `--timeout`, or `Ctrl+C` – the script sends `SIGINT` to the *entire process group* (`os.killpg`), waits up to 5 seconds for a graceful exit, and escalates to `SIGKILL` only if that fails. This avoids leaving an orphaned CMSITminiDAQ process running in the background after the monitoring script itself has exited.

The flag `--no_launch` exists specifically to decouple these two responsibilities when something else is driving Ph2_ACF's lifecycle, letting `log_to_csv.py` run purely as a passive LOG parser, or also in the case of processing old LOG file (see appendix C).

7.2 Run-number lifecycle management

`RunNumber.txt` is Ph2_ACF's own record of “which run is next.” The script reads it once *before* monitoring starts purely as a baseline, then re-reads it *after* monitoring ends. If the value is unchanged the script increments it itself; if it has been changed by Ph2_ACF the script leaves it alone. This makes the script safe to many consecutive runs without either getting stuck re-reading a LOG of a finished run.

7.3 Real-time LOG parsing

The core loop in `process_live_log()` reads the tail of the LOG file – `readline()` in a loop, and when it hits the current end of the file (`not line`), it waits and retries rather than treating that as EOF, since Ph2_ACF may still be writing more. An inactivity `--timeout` declares the run genuinely finished if no new line has appeared for that many seconds – this is what lets the

script stop automatically if `Ph2_ACF` is interrupted without the `LOG` ever reaching its normal end marker.

Because a single `LOG` line only carries a *fragment* of context (a register’s value, but not which board/chip/hybrid it belongs to), the parser is necessarily **stateful**: it tracks the “active hierarchy” (which board/optical-group/hybrid/chip the *subsequent* lines refer to) as it reads, updated whenever it encounters a context-header line (`Reading monitored data for [board/opticalGroup/hybrid/chip = ...]`). Every following telemetry line is then tagged with that hierarchy until the next header line changes it. This state machine also separately tracks:

- **Link type** (optical vs. electrical) detected once at the start of the `LOG` and deciding whether LpGBT-specific parsing applies or not.
- **eFuse IDs** for both LpGBT chips and CROC chips – these are unique hardware identifiers announced early in the `LOG`, and then need to be attached to every subsequent measurement row for that chip. They are captured into `efuse_map/temp_chip_efuses` dictionaries linking them to the hierarchical structure of the setup (`board/opticalGroup/hybrid/chip`), used for identifying the monitored chip.

7.4 CROC vs. LpGBT: two different telemetry shapes

The `LOG`’s hierarchy depth tells the parser which *regex-parsing* path to use: a 4-level hierarchy (`board/opticalGroup/hybrid/chip`) means CROC data, parsed by `CROC_telemery()` against the `CROC_REGISTERS` list; a 2-level hierarchy (`board/opticalGroup`) means LpGBT data, parsed by `LpGBT_telemery()` against the `ADC/voltage` register lists. A LpGBT reading logically applies to *every* CROC chip connected to that optical group, so the code duplicates the row once per known chip (`matching_chips`) rather than storing it once – in this way that `exporter.py` never needs to know about this one-to-many relationship; every row it reads is already self-contained.

7.5 Unit conversion at the point of ingestion

Two conversions happen inline, before a row is ever written to disk:

- `croc_register_conversion()` applies fixed multiplicative factors (e.g. $VINA \times 4$) that come from the CROC known resistor-divider values [2, Ch. 15.5].
- `convert_lpgbt_adc() / adc_raw_to_voltage()` apply a *per-chip* calibration, loaded once at startup from a CSV file (`load_lpgbt_calibration()`) keyed by each chip’s eFuse, converting a raw ADC count into a calibrated voltage using that chip’s individually-measured slope/offset (and, optionally, a temperature correction term). This only fires for the specific ADC channels on specific chip variants known to lack their own on-chip calibration (`PROBLEMATIC_LPGBT_ADC_REGISTER`, in `csv_schema.py`); everything else passes through unconverted [4, Ch 13.7].

Doing this conversion here, rather than downstream, means every later stage of the pipeline (as Prometheus and Grafana) only ever deals in final physical units.

7.6 Duplicate suppression

Because the same measurement can occasionally appear more than once within the same logged second (an artifact of the `LOG` format), `seen_this_second` is a set of already-written row, cleared every time the observed timestamp advances. With this de-duplication two identical rows within the same second are collapsed to one.

7.7 Output: one append-mode CSV, not a chunked writer

Despite the possibility given by `--output_csv_name` to specify a precise name for the Full CSV file produced by `log_to_csv.py` (or even use the run number for it), the default settings of the code don't do any chunking or rotation, but with `open_full_csv_file()` just open the target CSV in append mode (writing a header only if the file is currently empty) and every batch of new rows is written and immediately flushed. This is deliberate: it keeps this code's only responsibility "turn LOG lines into correctly-formed CSV rows", which is exactly the responsibility that disappears if Ph2_ACF is ever changed to emit the Full CSV natively.

8 `csv_chunker.py`

Purpose: stage 2. Bridges the "one continuously-growing Full CSV" produced by stage 1 into the "many small chunk files" that stage 3 needs, while keeping the Full CSV's disk footprint bounded.

8.1 Tailing the Full CSV safely

`tail_and_reduce()` opens the Full CSV in read/write mode and follows the same `readline()` + retry-on-EOF pattern as `log_to_csv.py`'s LOG tailing, reading only complete, newline-terminated lines and remembering its file position between iterations.

8.2 Chunking by monitoring cycle, not by size or time

The key design decision in `ChunkWriter` is `_get_signature()`: it derives an identity for a row from every field *except* the ones that legitimately vary on every reading (VALUE, ERROR, DATE, TIME). The very first row written into a fresh chunk (`first_row_signature`) has its signature remembered; every subsequent row is compared against it, and the moment a row with that *same* signature reappears, the chunker knows a full monitoring cycle has completed – Ph2_ACF has cycled back around to reading that same register again – and closes the current chunk right there. This means chunk boundaries align with physically meaningful units (one complete pass over all monitored registers) rather than an arbitrary time or row-count threshold that might split a single cycle's readings across two chunks.

Chunk files follow this pipeline: lines are written into a `.csv.tmp` path by `tail_and_reduce()` while the chunk is being read (from the Full CSV), then renamed to a `.csv` extension only once the chunk is complete. The rename is important: the `exporter.py` (which only ever globs for `*.csv`) can never observe a half-written file (`.csv.tmp`). Orphaned `.csv.tmp` files from a previous, ungracefully-terminated session are swept, since nothing else would ever clean them up otherwise.

8.3 Bounding the Full CSV's size, without racing the writer

This is the part requiring the most care, because two processes share the same Full CSV file: `log_to_csv.py` writing and `csv_chunker.py` reading *and* periodically truncating it. The safety logic:

1. Truncation is only even considered once the file has genuinely stopped growing for this read pass (i.e. inside the `not line / EOF` branch) – never mid-read.
2. It is gated on a second, independent condition: `chunk_dir` must currently contain **zero** `*.csv` files – meaning `exporter.py` has already fully drained every chunk produced so far. If it has not, truncation is skipped this pass and retried later. This prevents ever discarding data that has not reached Prometheus yet.

3. When both conditions hold, the truncation itself is done carefully: any data written to the file *after* the last successful read but *before* the truncation (`unread_data = f.read()`) is captured and rewritten back immediately after truncating, so nothing written in that narrow window is lost. `f.truncate(0)` shrinks the file, and only the header plus that just-captured unread data is written back.

- `--max_mb` controls how large the Full CSV is allowed to grow before this kicks in.

Optional debug archive - `--debug_csv`, when given, opens a second file in permanent append mode (never truncated) and mirrors every row into it in parallel with the normal chunking path. This exists purely for offline inspection/debugging – a complete, un-reduced history of a run, independent of whatever chunking or truncation the live pipeline is doing.

9 exporter.py

Purpose: stage 3. The only stage that actually speaks Prometheus’s protocol. Implements a custom `prometheus_client` collector that watches the chunk directory and exposes the latest known value for every distinct monitored quantity as a single Gauge metric, differentiated by label.

9.1 The label set is the identity of a time series

`ChunkCollector` builds a 10-element tuple (board, optical group, portcard ID, LpGBT eFuse, hybrid, chip, chip eFuse, module ID, register, unit) from each row and uses that tuple as the dictionary key for `latest_data`. This tuple is exactly what becomes the Prometheus label set on `hardware_monitoring_live_value`, meaning Grafana can filter/group by any combination of these – e.g. “show LPGBT_TEMPERATURE across all hybrids for a specific board” – without needing separate metric names per register.

9.2 Consume-then-delete, on every scrape

`_consume_chunks()` runs at the start of every `collect()` call (i.e. every time Prometheus scrapes /metrics): it globs and parses the `*.csv` files in the chunk directory, merges every row into `latest_data/last_seen`, and deletes the file afterward. This is what keeps the chunk directory from growing without bound: nothing else in the pipeline is responsible for cleaning it up.

9.3 Per-series staleness, not global staleness

This is a piece of non-obvious logic, and it is worth explaining the reasoning rather than just the mechanism: different registers are read at very different real-world cadences (some registers cycle every few seconds, others take much longer per reading). If staleness were tracked with one global “last update” timestamp for the whole collector, a fast-updating register would keep that timestamp fresh forever, masking the fact that a *different*, slower register had actually stopped updating (e.g. because that specific chip failed) – it would sit frozen at its last known value indefinitely instead of correctly disappearing.

The fix implemented here is to track **last_seen per label tuple**, not globally. On every `collect()`, each series is checked independently against `--stale_after`, and only *that* series is pruned if it has gone quiet – completely independent of whether every other series is still updating normally. This means that a dead sensor correctly vanishes from Grafana on its own schedule, while a healthy one two rows away is entirely unaffected.

- `--stale_after` needs to be set comfortably above the *slowest* register’s real read interval – set too low, a register that is actually fine gets spuriously pruned every cycle (visible in Grafana

as periodic gaps); set appropriately, it only ever fires for a genuinely stalled series. The default `stale_after` parameter is set to 150 seconds.

10 Other Applications

This code can be used also if the orchestration of the process (launching `CMSITminiDAQ`) is not performed by `log_to_csv.py`. For example during the project, I explored the monitoring of the `LpGBT_ADC7`, which for all the LpGBT is related to the measurement of the VTRX+ optical power (RSSI). I tried to understand if there were a correlation between the value of some VTRX+ registers and the corresponding monitored value. To understand the correlation is useful to launch a particular calibration: VTRX+ Light Yield Scan – `vtrx`. During this calibration two parameters related to the VTRX+ are changed: `VTRxBias` and `VTRxModulation` (see `Ph2_ACF` User Guide for more details). Launching different calibration changing each time the `VTRxBias` and `VTRxModulation` extremes, and monitoring the `LpGBT_ADC7` register before and after the calibration – in principle – one should see the correlation.

For implementing this, I wrote a code that changes the `VTRxBias` and `VTRxModulation` values in the XML configuration file, and launches the calibration. I embedded in this code also the parsing of the monitored values, using the code `log_to_csv.py`. At each run, the LOG file is parsed and the CSV is filled. Saving also the run number, I could establish or reject any correlation between what monitored in the `LpGBT_ADC7` and what actually modified by the calibration.

Anyway, any the correlation explored in this way has been rejected. The reason is not because `LpGBT_ADC7` is insensitive to RSSI, but actually because with the `vtrx` calibration this register value is not affected (monitored only the upstream link, but the calibration changed the downstream one).

11 Results: Grafana Plots

In this section some of the Grafana plots obtained in this project are presented.

Figure 10: An example of the temperature evolution for the Setup 1 chips ($2 \times \text{CROC} + 1 \times \text{LpGBT}$). Only the registers monitoring the temperature are highlighted. Notice that in some point the lines are not connected, because a `Ph2_ACF` run was stopped and then another one started.

Figure 11: An example of the temperature evolution for the Setup 2 chips ($3 \times \text{LpGBT}$). Only the registers monitoring the temperature are highlighted. The connection LpGBT-eFuse can be seen in the `csv_schema.py`: for all the LpGBTs `LPGBT_ADC6` measures the VTRX+ temperature, and, for the central LpGBT, `LPGBT_ADC0` measures the PortCard temperature. Notice that ADC0 and NTC sensor for the middle LpGBT has a very similar behavior, while ADC6 is quite different – that could be explained by an error in the calibration of one of the register.

Figure 12: An example of the voltages evolution for the Setup 2 chips ($3 \times \text{LpGBT}$). The connection LpGBT-eFuse can be seen in the `csv_schema.py`: `LPGBT_ADC2` measured the VDD1V2 (around 1.25 V for VTRX+ powering), `LPGBT_ADC3` measured the VDD2V5 (around 2.5 V for LpGBT powering) and `LPGBT_ADC4` measured the V_IN (around 10 V, the PortCard input voltage).



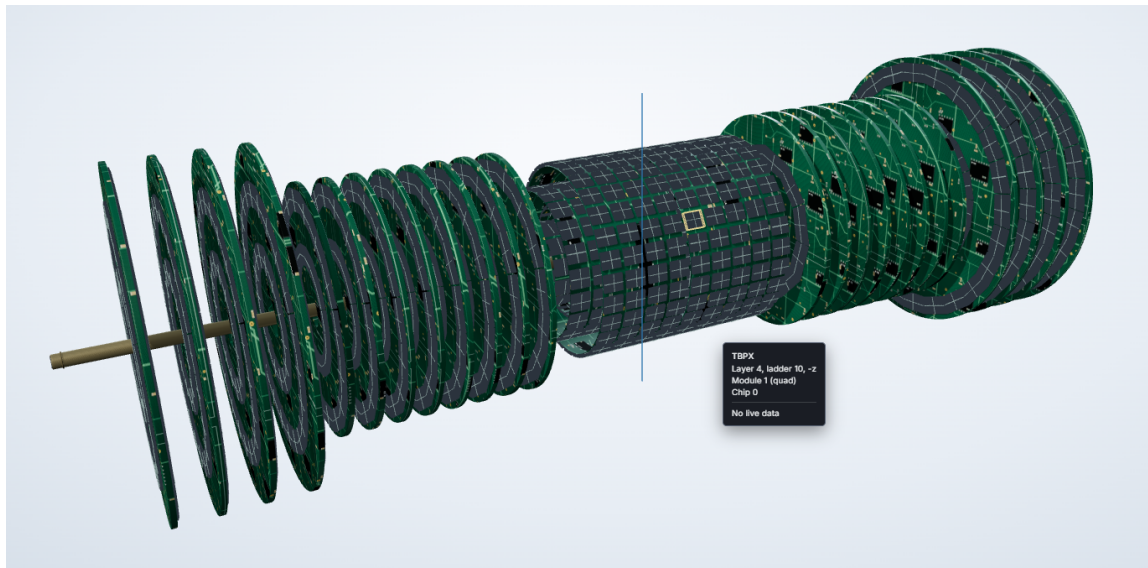
Figure 10



Figure 11



Figure 12

Figure 13: 3D structure of Phase2 - Inner Tracker and monitoring live values per chip. *Image courtesy of Athar Ahmad.*

A CSV Data Sizing

Quick analysis of the CSV file sizes based on laboratory setup (as shown in section 3):

- **Average Record Size:** ≈ 72 bytes per CSV line.
- **Setup 1** ($2 \times \text{CROC} + 1 \times \text{LpGBT}$): ≈ 60 lines/cycle $\implies \approx 4.3$ kB/cycle.
- **Setup 2** ($3 \times \text{LpGBT}$): ≈ 48 lines/cycle $\implies \approx 3.5$ kB/cycle.
- **Setup 3** ($2 \times \text{LpGBT}$): ≈ 32 lines/cycle $\implies \approx 2.3$ kB/cycle.

B Prometheus YAML file

For this project has been used the standard version of Prometheus, whit this YAML file. Notice in particular the `scrape_interval: 1s`, each second Prometheus looks to new metrics to scrape. This guarantees the possibility to scrape the data and not loosing anything even if only few registers are monitored and the `MonitoringSleepTime` set in the XML file is short.

```
global:
  scrape_interval: 1s
  evaluation_interval: 1s
scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
        labels:
          app: "prometheus"

  - job_name: "csv_exporter"
    scrape_interval: 1s
    static_configs:
      - targets: ["127.0.0.1:8000"]
```

C Off-line LOG Parsing

`log_to_csv.py` can also be used in not-live parsing (for example to produce a CSV file from old LOG files). But this need careful operations. As explained in appendix B, the native code pipeline is for live-monitoring, and for this reason Prometheus YAML file as no ossibility of looking in the past, but only assign a specific time (the instant of scraping). To implement old-LOG file monitoring you need:

- To be specified into the `log_to_csv.py` (via `monitoring.ini` or CLI):
 - `--run`: a specific value (to search for the associated already completed LOG file).
 - `--date`: the run date (to be inserted into the appropriate CSV field).
 - `--log_time`: flag to indicate to use for time CSV field, the one printed in the LOG file.
- Use a specific version of the `exporter.py` that take as time the one in the CSV column, and not the instant of scraping.
- Modify the Prometheus YAML file adding this lines at the end, considering the period to go back in time to be able to add this data:

```
storage:
  tsdb:
    out_of_order_time_window: <period_to_go_back_in_time>
```

D GitHub Repository

The all code can be found at this GitHub repository:

https://github.com/BelottiMarco/CMSIT_monitoring

References

- [1] CMS collaboration et al. “Evaluation of 3D pixel silicon sensors for the CMS Phase-2 Inner Tracker”. In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* (2026), p. 171359.
- [2] Maurice Garcia-Sciveres, Jorgen Christiansen, and Flavio Loddo. *RD53B manual*. Tech. rep. CERN, 2019.
- [3] Giulio Bardelli and CMS Tracker Group. “Electrical measurement and read-out performance of a realistic, full-scale system bench of the CMS Inner Tracker Barrel for HL-LHC”. In: *Journal of Instrumentation* 20.03 (2025), p. C03031.
- [4] Jose Fonseca et al. *LpGBT documentation: release*. Tech. rep. CERN, 2022. URL: <https://lpgbt.web.cern.ch/lpgbt/v2/>.
- [5] CMS Tracker Group. *Ph2_ACF User Guide*. 2026. URL: <https://ph2acf.docs.cern.ch/>.